KP19 The Nonlinear SVM and the Mercer Kernel Trick

> Level: undergraduate, year 3
>
> Pattern Recognition | Chapter 4, §4.18

Slides: read and download

Learning objectives

- Explain that the SVM rule uses only inner products and state the inner-product identity of the quadratic map.
- State Mercer's theorem and what it does not tell us, and write the dual problem and the rule of the kernelized SVM.
- Use experimental numbers to explain the choice of kernel, the effect of the hyperparameters γ and C, and the cost and the limit of the kernel trick.

KP19 The Nonlinear SVM and the Mercer Kernel Trick

One-sentence summary: The SVM computes with inner products only, so replace the inner product by a kernel function; we then design a linear classifier in a high-dimensional space that we never visit, and the cost does not depend on the dimension of that space.

1. Inner products only

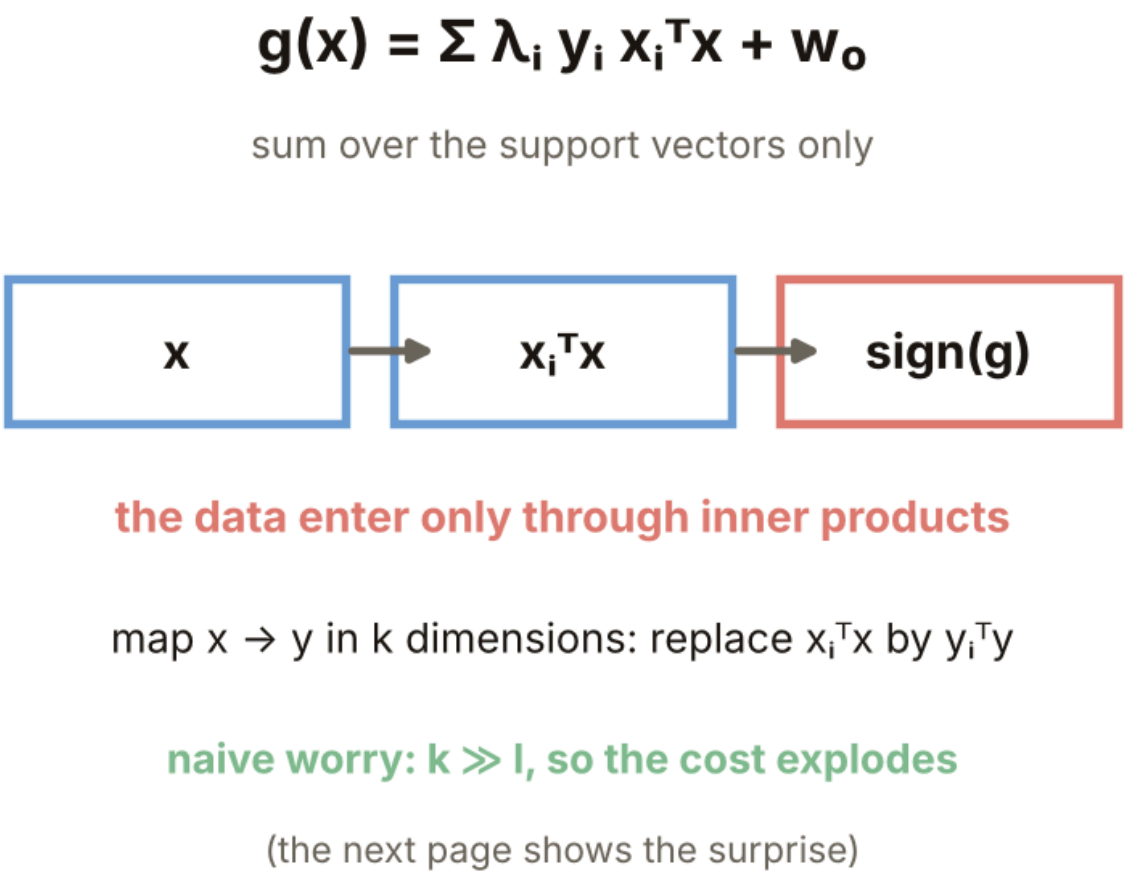
Plain explanation: Is there a way to keep only a few points and still classify nonlinearly?

- **Support vectors**: the training points whose weight $\lambda_i > 0$ after training; the rule depends on them alone, and there are usually far fewer than all the training points.

--

The rule is $g(x) = \sum_{i=1}^{N_s} \lambda_i y_i \, x_i^T x + w_0$ with x of dimension l and N_s support vectors; the data enter only through the inner product $x_i^T x$. To design in a k-dimensional mapped space, replace the vectors by their images y and the inner product by $y_i^T y$. The naive worry: $k \gg l$, so surely the cost explodes?

Figure `lec05-kp19-inner`: the flow of the rule, with the data entering only through inner products.



2. A quadratic map

Kernel function $K(x, z)$: a function that gives the mapped inner product $\phi(x)^T \phi(z)$ directly, without computing the map first. **Example**: for $y = [x_1^2, \sqrt{2}x_1x_2, x_2^2]^T$, $y_i^T y_j = (x_i^T x_j)^2$. Check: $x_i = (1, 2)$, $x_j = (2, 1)$: $y_i = (1, 2.828, 4)$, $y_j = (4, 2.828, 1)$, inner product $4 + 8 + 4 = 16$; in the original space $(x_i^T x_j)^2 = 4^2 = 16$, the same. The high-dimensional inner product comes from the original-space inner product, so the entry fee is zero.

Figure `lec05-kp19-identity`: the mapped and the original inner product of two vectors.

$$y = [x_1^2, \sqrt{2} \cdot x_1 x_2, x_2^2]$$

$x_i = (1, 2)$ $\quad\quad$ $y_i = (1, 2.828, 4)$

$x_j = (2, 1)$ $\quad\quad$ $y_j = (4, 2.828, 1)$

high-dimensional: $y_i^T y_j = 4 + 8 + 4 = 16$

original space: $(x_i^T x_j)^2 = (4)^2 = 16$

the same number, and 3-D was never visited

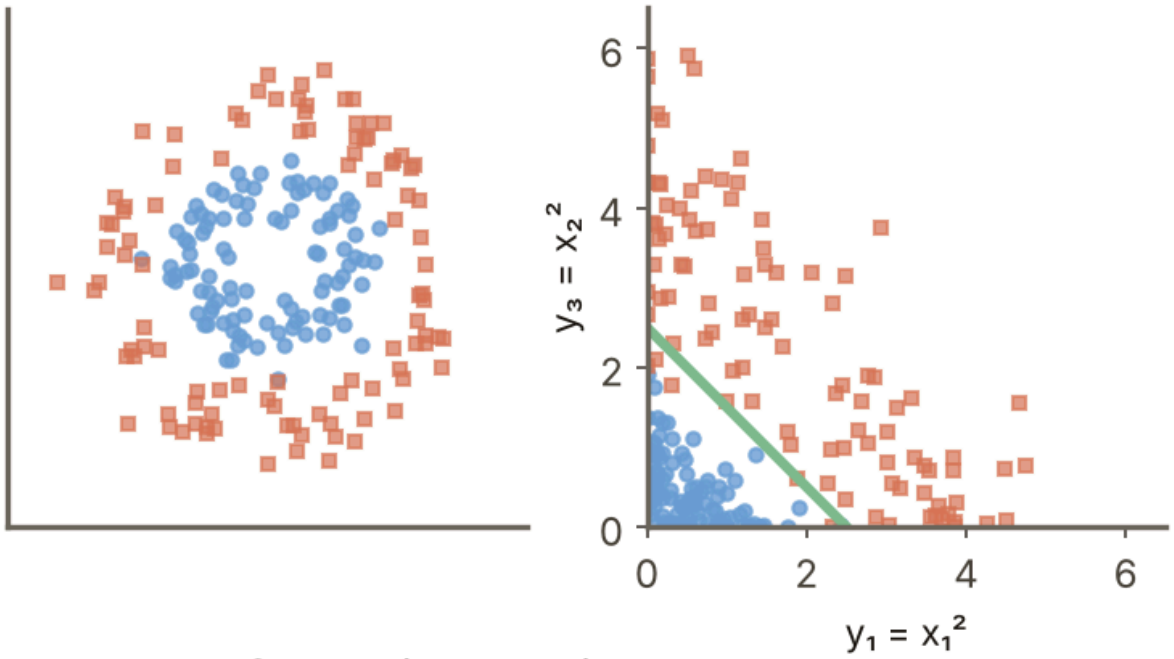
$$y_i^T y_j = (x_i^T x_j)^2$$

entry fee of the high-dimensional space: zero

3. Lifting the rings

Example (designed here): two concentric rings of radius about 1 and 2 (sd 0.25), 60 training and a separate 100 new points per class. After the map $y_1 + y_3 = x_1^2 + x_2^2 = \|x\|^2$, so the map turns the radius into a coordinate; the line $y_1 + y_3 = 2.5$ cuts between the rings. One cut gets 97.5% of 200 new points (100 per class) right, a few overlapping at the edges; a plane in 3-D is a circle in the original space. The figure leaves out the middle coordinate $\sqrt{2}x_1x_2$.

Figure `lec05-kp19-lift`: the rings on the left, and on the right the coordinates y_1 and y_3 after the map.



left: two rings, no line separates them

right: after the map the line $y_1 + y_3 = 2.5$ cuts them

$y_1 + y_3 = \|x\|^2$, one cut gives 97.5% on these 200 points

(the middle coordinate $\sqrt{2}\cdot x_1 x_2$ is not drawn)

4. Mercer's theorem

Mercer condition: K is symmetric and continuous and $\iint K(x,z)g(x)g(z)\,dx\,dz \geq 0$ for every square-integrable g. **Formal definition (theorem)**: a K meeting the condition has a space H (called an RKHS) and a map ϕ with $K(x,z) = \langle \phi(x), \phi(z) \rangle$, and conversely. What the theorem does not say: what ϕ looks like or the dimension of H; the Gaussian kernel's H is infinite-dimensional. A finite check (added): the Gram matrix $[K(x_i, x_j)]$ must be positive semidefinite; for 30 random points the smallest eigenvalue is 8.8×10^{-8} for the Gaussian kernel and -8×10^{-15} for the quadratic one (rounding, taken as 0).

Figure `lec05-kp19-mercer`: the content of the theorem, what it leaves unsaid and the smallest Gram eigenvalue of two kernels.

x → φ(x) in a space H

K(x, z) = ⟨φ(x), φ(z)⟩

exists if K is symmetric, continuous and

∫∫ K(x,z) g(x) g(z) dx dz ≥ 0 for every g

what the theorem does NOT tell you

what φ looks like, or the dimension of H

(the Gaussian kernel's H is infinite-dimensional)

30 random points, smallest eigenvalue of the Gram matrix

Gaussian (γ = 0.5)	8.8e-08
quadratic $(x^Tz + 1)^2$	-8.1e-15

5. The kernelized SVM

Common kernels: polynomial $(x^T z + 1)^q$; Gaussian $\exp(-\gamma\|x - z\|^2)$; hyperbolic tangent $\tanh(\beta x^T z + \gamma)$ (may be illegal, see the extension). The dual problem:

$$\max_{\lambda} \sum_i \lambda_i - \tfrac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j K(x_i, x_j), \quad 0 \le \lambda_i \le C, \sum_i \lambda_i y_i = 0,$$

where C caps the penalty for violations. The rule is $g(x) = \sum_{i=1}^{N_s} \lambda_i y_i K(x_i, x) + w_0$, and $g > 0$ goes to ω_1. Check (added): with the solved λ this formula differs from the library's decision value by at most 10^{-14}. Because the kernel is nonlinear, this is a nonlinear classifier in the original space.

Figure `lec05-kp19-kernels`: the three common kernels, the dual problem and the rule.

polynomial

$$K(x, z) = (x^T z + 1)^q$$

Gaussian (RBF)

$$K(x, z) = \exp(-\gamma \|x - z\|^2)$$

hyperbolic tangent

$$K(x, z) = \tanh(\beta x^T z + \gamma)$$

$$\text{dual: } \max \Sigma \lambda_i - \tfrac{1}{2} \Sigma \lambda_i \lambda_j y_i y_j K(x_i, x_j)$$

$$0 \le \lambda_i \le C, \quad \Sigma \lambda_i y_i = 0$$

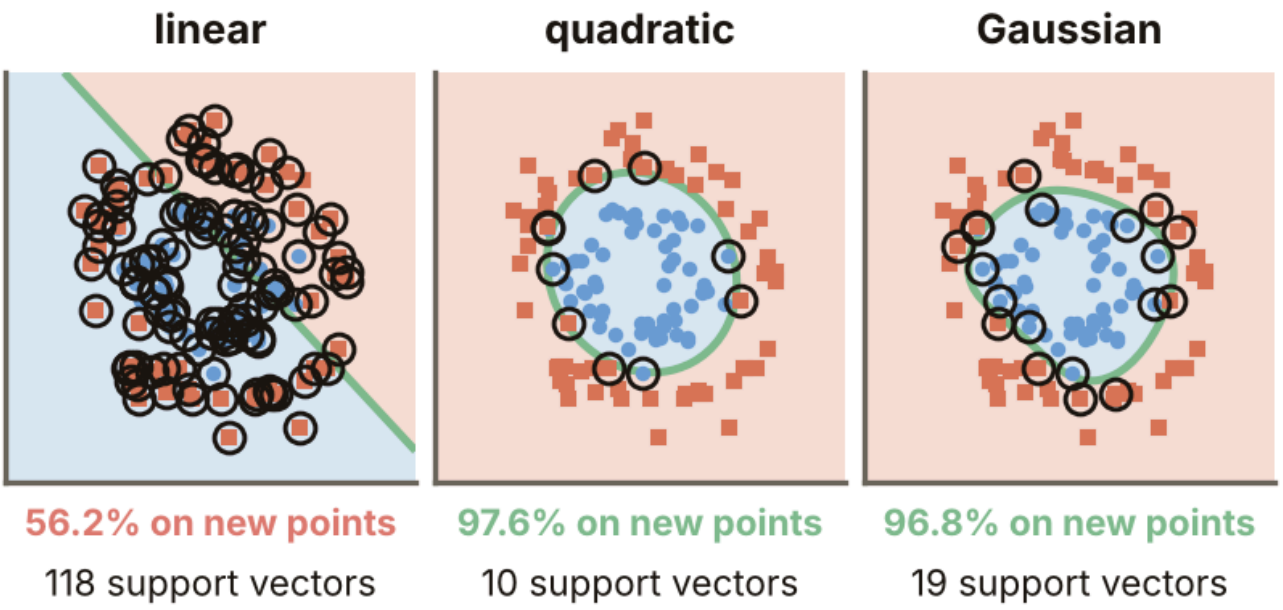
$$g(x) = \Sigma \lambda_i y_i K(x_i, x) + w_0$$

nonlinear in the original space, linear in H

6. The rings experiment

Example (designed here): concentric rings, 60 training and 500 new points per class; $C = 10$, Gaussian $\gamma = 0.5$, quadratic kernel $(x^T z + 1)^2$. Accuracy on new points: the linear kernel 56.2% with 118 support vectors; the quadratic 97.6% with 10; the Gaussian 96.8% with 19. Same data and algorithm, only the kernel changes, from near guessing to about 97%; the Gaussian kernel keeps 19 support vectors where a PNN keeps all 120 training points (compare KP21).

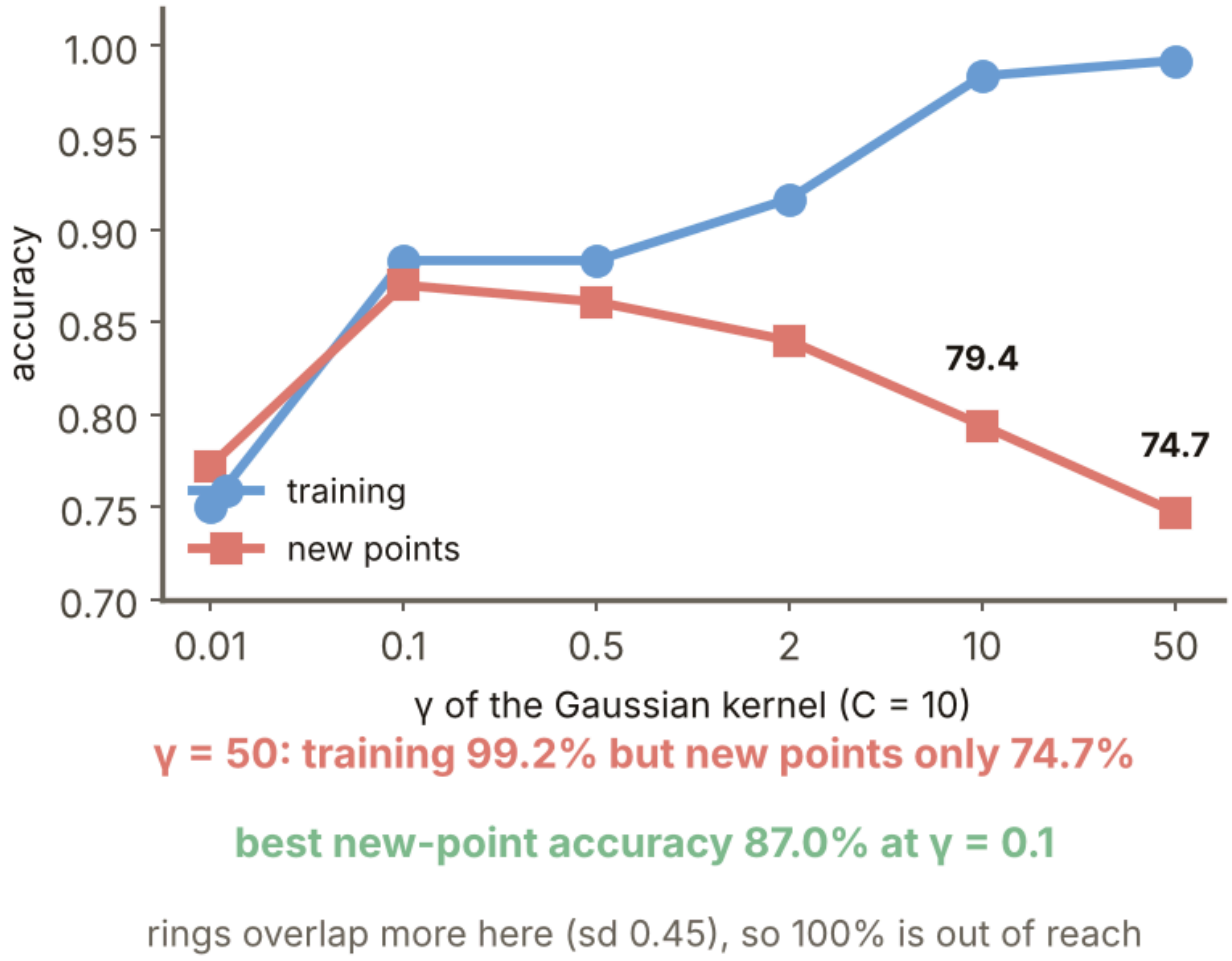
Figure `lec05-kp19-regions`: the decision regions and support vectors (circles) of three kernels.



7. Hyperparameters

Hyperparameter: a parameter set before training rather than solved from the data, here the Gaussian γ and the penalty cap C. **Example** (designed here): rings that overlap more (sd 0.45), $C = 10$, sweeping γ. Accuracy on training and new points: $\gamma = 0.1$: 88.3%, 87.0% (best); $\gamma = 10$: 98.3%, 79.4%; $\gamma = 50$: 99.2%, 74.7%. Sweeping C at $\gamma = 0.5$, new points: $C = 0.01$: 84.3%; $C = 1$: 86.4% (best); $C = 1000$: 82.8%. The best values were picked on the new points; in practice use separate validation data. **Common misconception**: the higher the training accuracy the better; when γ is too large the training fit tightens and the new points get worse.

Figure `lec05-kp19-hyper`: training and new-point accuracy against the Gaussian γ.



8. Dimension-free

A polynomial kernel of dimension l and degree q has $\binom{l+q}{q}$ features: $(l, q) = (2, 2)$: 6; $(10, 3)$: 286; $(100, 5)$: 96,560,646; $(784, 5)$: 2,515,874,132,157 (about 2.5×10^{12}). One kernel value is an inner product and a power, about l multiplications whatever the kernel-space dimension; the infinite-dimensional Gaussian space still costs one distance. **Limit**: training still needs an $N \times N$ Gram matrix, and nobody knows how to pick the best kernel. **To remember**: kernel trick = any algorithm that uses only inner products gets higher dimensions for free; the price is not knowing which space you are in.

Figure `lec05-kp19-cost`: explicit features of a polynomial kernel against the multiplications of one kernel value.

dim I	degree q	explicit features	kernel
2	2	6	2 mult.
10	3	286	10 mult.
100	5	96,560,646	100 mult.
784	5	2,515,874,132,157	784 mult.

explicit features = C(I + q, q); one kernel value = one inner product

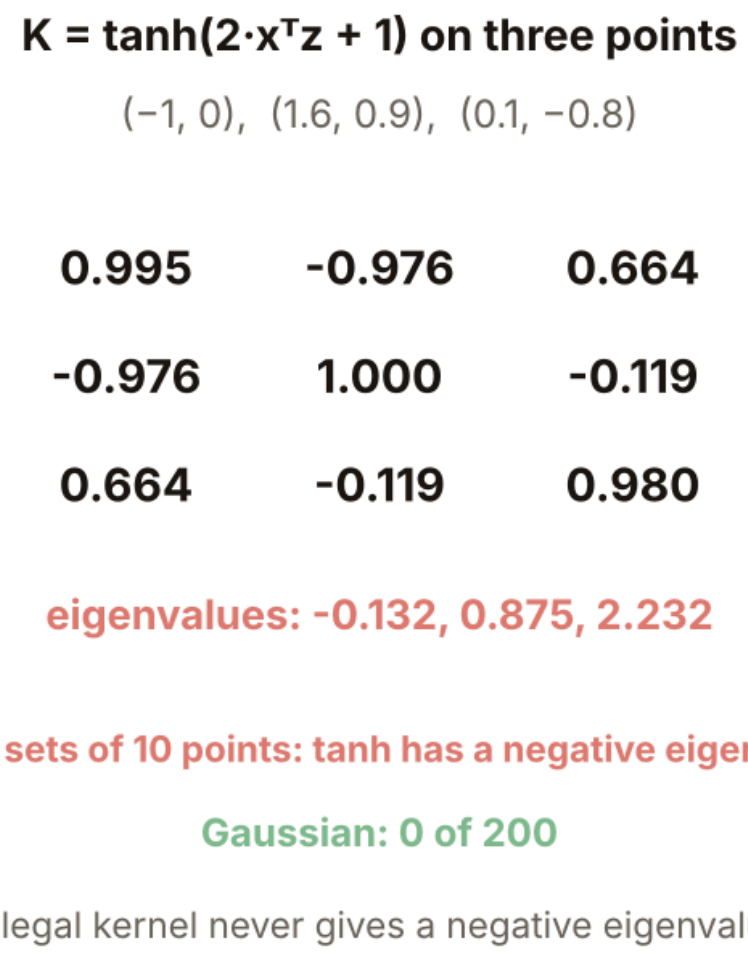
cost does not depend on the dimension of the kernel space

(it still grows with the number of training points N)

Extension: an illegal kernel, practice and worked solution

The tanh kernel can be illegal (designed here): $K = \tanh(2x^T z + 1)$ on the three points $(-1, 0)$, $(1.6, 0.9)$, $(0.1, -0.8)$ gives a Gram matrix with eigenvalues -0.132, 0.875 and 2.232; the negative one violates the Mercer condition. With 10 random points drawn 200 times the tanh kernel has a negative eigenvalue in all 200 draws and the Gaussian kernel in none. The hyperbolic tangent with $(\beta = 2, \gamma = 1)$ is not a legal kernel and the dual problem need not be convex (a theoretical inference, not measured here); check before using it, or use the Gaussian kernel.

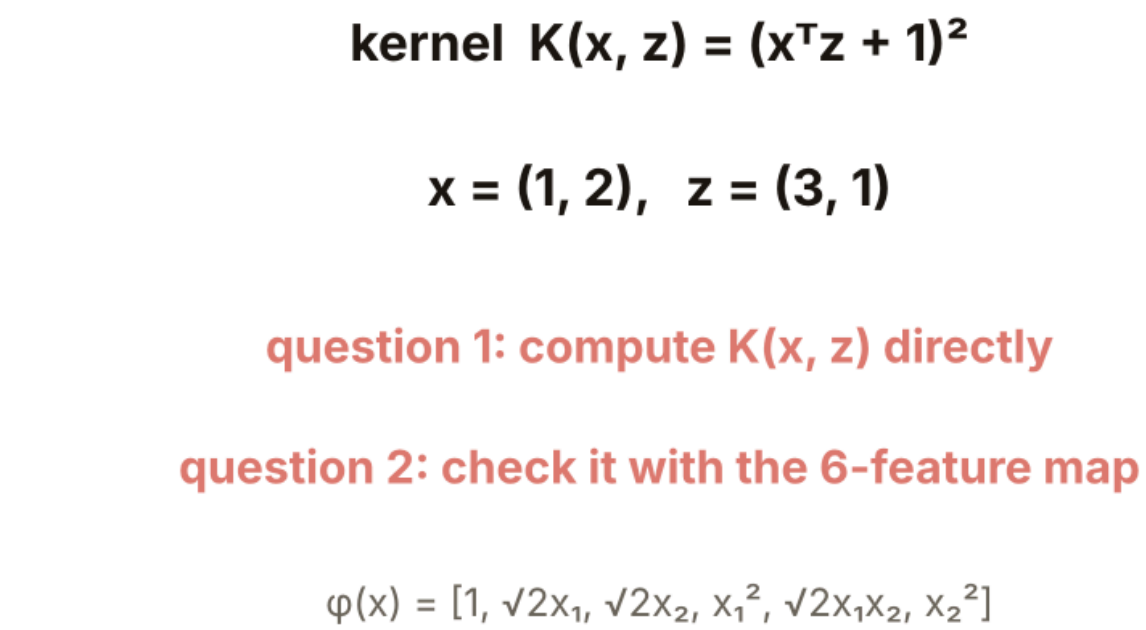
Figure `lec05-kp19-tanh`: the Gram matrix and eigenvalues of three points.



Practice: $K(x, z) = (x^T z + 1)^2$, $x = (1, 2)$, $z = (3, 1)$; the map $\phi(x) = [1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2]$.

1. Compute $K(x, z)$ directly with the kernel.
2. Compute the 6 features $\phi(x)$ and $\phi(z)$, take their inner product and check it against question 1.

Figure `lec05-kp19-practice`: the setup of the practice.



Worked solution of questions 1 and 2: (1) $x^T z = 1 \cdot 3 + 2 \cdot 1 = 5$ and $K = (5 + 1)^2 = 36$, with only 2 multiplications. (2) $\phi(x) = (1, 1.414, 2.828, 1, 2.828, 4)$ and $\phi(z) = (1, 4.243, 1.414, 9, 4.243, 1)$; the inner product is $1 + 6 + 4 + 9 + 12 + 4 = 36$, the same as question 1. (3) Check: the same 36; the kernel works in 2-D while the map first builds 6 features, and the gap widens with dimension.

Figure `lec05-kp19-practice-sol`: the computation both ways.

question 1: $x^T z = 3 + 2 = 5$

$$K = (5 + 1)^2 = 36$$

question 2: $\varphi(x) = (1, 1.414, 2.828, 1, 2.828, 4)$

$\varphi(z) = (1, 4.243, 1.414, 9, 4.243, 1)$

$$\varphi(x)^T \varphi(z) = 1 + 6 + 4 + 9 + 12 + 4 = 36$$

same value 36; the kernel needed 2 multiplications, the map 6 features

Terms and source

中文	English
支撐向量	Support vector
核函數	Kernel function
Mercer 條件	Mercer condition
超參數	Hyperparameter

Source: Chapter 4, KP19, §4.18 (read 2026-10-25). That the SVM uses only inner products, the quadratic-map identity, Mercer's theorem and what it does not say, the kernelized dual and rule, the complexity independent of the kernel-space dimension and the 'to remember' line follow the source; the rings experiment, the sweeps of γ and C, the feature-count table, the negative eigenvalues of the tanh kernel and the practice were designed and recomputed for this lecture ([added], (program-computed)).